1 week from 16 feb 2026(Worked on the Idea and

submitted to the professor)

Project Proposal: AI Powered Code Navigator

Introduction

I am working on a project called **AI Powered Code Navigator**, which is basically a tool to help developers understand big codebases faster. Many times when we join a new project or look at some open source repo, it takes a lot of time to figure out *where* specific logic is written, like authentication, API routes, or bussiness rules.The idea of my project is to let the user **upload code files or give a GitHub repo URL**, then the system will read the code, store it, and use **AI (LLM)** so that the user can ask questions in normal English, like:

- "Where is user login handled?"
- "Which file defines the booking API?"

Instead of searching manually through hunderds of files, the user get's a direct answer based on the code that was uploaded. The main goal is to make code navigation smoother, faster and more user-friendly, specially for new developers on a project.

Features

Code Ingestion (Upload & GitHub)

- **File Upload:**
- User can upload multiple source code files (right now focusing on .js files).
- The backend reads the content of each file and prepares it for processing.
- **GitHub Repo Import:**
- User can paste a GitHub repository URL, like https://github.com/user/repo.
- The backend downloads the repo as a ZIP file, extracts it and scans through the files to pick the relevant ones.

Automatic Code Summarization

- For each file, the system:
- Sends the code to the **OpenAI API**.
- Gets back a **short summary** that explains what that file is doing in simple words.

- These summaries are saved in the database together with the code, so later we don’t need to call the API again for the same file.

Natural Language Code Search

- The user can ask questions in plain English, for example:
- “Where do we validate JWT tokens?”
- “What does the user controller do?”

Storage of Code and Metadata

- Processed files are stored in **MongoDB** with:
- fileName
- filePath (for uploaded files)
- code
- summary

  This makes it easy to reuse past uploads and also opens the door for future things like per-project IDs or embeddings.

Technical Implementation

Backend

- **Node.js + Express**:
- Used to build the REST API.
- Routes for: will decide

Database

- **MongoDB** (with **Mongoose**):
- Stores each code file as a document.
- Simple schema: fileName, filePath, code, summary.
- This schema is flexible and can be extended later (for example projectId or embedding field).

File & Repo Handling

- **multer**:
- Handles multipart form uploads from the client.

- Temporarly saves files in an uploads/ folder.
- **adm-zip**:
- Used to read the ZIP file downloaded from GitHub.
- Iterates over entries and only processes .js files for now.
- **Octokit (@octokit/rest)**:
- GitHub API client.
- Downloads the repository as a ZIP using owner + repo extracted from the URL.

AI Integration

- **OpenAI Node SDK**:
- generateSummary(code):
- Calls a chat model (like gpt-4o-mini) with a prompt: “Summarize the following code”.

Tech Stack (Summary)

- **Languages**: JavaScript (Node.js)
- **Backend Framework**: Express.js
- **Database**: MongoDB + Mongoose
- **File Uploads**: multer
- **ZIP Handling**: adm-zip
- **GitHub Integration**: Octokit (@octokit/rest)
- **AI / LLM**: OpenAI API (chat completions)

Conclusion

AI Powered Code Navigator is my attempt to take common software engineering pain **finding and understanding code in large projects** and solve it using modern backend development and AI tools. The project shows how to combine:

- API design,

- external services (GitHub, OpenAI),
- database storage (MongoDB),

into one practical system. In the future, I plan to improve it with features like embeddings-based search, better support for multiple projects, and maybe a nice React frontend with visualizations.

2 Week Progress(I have worked on the database schema and Intial endpoints for the project)

So the schema for the initial version for MongoDB will be like thwse

I'm using MongoDB to store the code and its summaries. Each file we process becomes one document in a collection. Right now the document has 4 fields:

- fileName – Name of the file (e.g. codecontroller.js).
- filePath – Path where the file was stored when the user uploaded it. We use this mainly for the file upload flow; for GitHub repo it might be empty or not used.
- code – The full source code of that file as text.
- summary – Short description of what the file does, generated by the AI (OpenAI).

So when we process a repo or some uploaded files, we create one such document per file and save it in MongoDB. Later when the user asks a question, we load these documents, take the code and summary, and send them to the AI to get an answer. We don't have a project or repo id yet, so all files from every upload go into the same "bucket" that's something I'd add later to separate different projects.

REST endpoints (what they do)

The backend has 4 main APIs, all under /api. Here's what each one does in simple words:

1. **POST /api/process-code**

- Use: When you want to process just one piece of code (e.g. paste a snippet) and get a summary.
- Send: JSON with code, fileName, and optionally filePath.

- Get back: The AI-generated summary for that code. We also save this one file in the database so it can be used in search later.

2. POST /api/search

- Use: This is the main “ask a question” API. User types something like “Where is login handled?” and we answer based on all the code we have stored.
- Send: JSON with query (the question in plain English).
- Get back: The AI’s answer (text) based on the stored codebase.

3. POST /api/upload

- Use: Upload multiple code files at once (e.g. 5–10 files). We read each file, get a summary from the AI, save everything in MongoDB, and then delete the temp files.
- Send: Form data with a list of files (we use “files” as the field name and limit to 10 files).
- Get back: A list of summaries, one per file (with file name and summary).

4. POST /api/upload-repo

- Use: User pastes a GitHub repo URL and we download the repo, take the JavaScript files, summarize each one, and save them in MongoDB.
- Send: JSON with repoUrl (e.g. https://github.com/username/repo-name).
- Get back: A success message and the list of files we processed (file name, code, summary for each).